Automated Equity Research - Project Guide

Repo: github.com/darrenye1/public-reports- | Site: public-reports-one.vercel.app | Educational demo only; not investment advice.

1. Project Overview

Single-repo pipeline: pull US equity data from Yahoo Finance (yfinance), run financial/industry/peer analysis, blend Simon FCFE DCF + comparable multiples + analyst targets, publish 2-page PDF reports and a Top 20 summary table, sync a static dashboard, and deploy via Vercel.

- Data: yfinance (Yahoo Finance)
- Outputs: reports/{TICKER}_research.pdf, Top20_Summary.pdf / .csv
- Website root: public/ (Vercel Output Directory = public)
- Automation: GitHub Actions every Monday + optional manual Run workflow

2. Directory Layout

```
equity_research.py Core: data, valuation, PDF, batch CLI (~3700 lines)
dashboard_data.py Dashboard macro news + index sparklines
sync_website.py Copy reports/ to public/ + docs/; build JS files
reports/ Generated PDFs/CSV; .data_state.json (gitignored)
public/ Deploy root: index.html, *.js, reports/
docs/ Mirror of public/ for GitHub Pages
.github/workflows/ weekly-update.yml - scheduled refresh
```

3. End-to-End Workflow

3.1 Fully automatic (recommended)

- Monday 14:00 UTC: GitHub Actions cron triggers
- python equity_research.py --batch-top20 --refresh-stale --replace-old
- python sync_website.py updates public/ and docs/
- Actions bot commits and pushes main; Vercel redeploys in ~1-2 min
- Prerequisite: Actions workflow permissions = Read and write

3.2 Local manual

```
python equity_research.py --batch-top20 --force --replace-old
sync_website.bat
GitHub Desktop: Commit + Push origin
```

3.3 Single ticker

```
python equity_research.py AAPL
python equity_research.py SPCX --export-excel
```

4. equity_research.py - Core Logic

4.1 Single-ticker pipeline

```
fetch_company(ticker)
-> build_financial_summary()
-> build_industry_research()
-> build_competitor_analysis()
-> build_valuation_summary()
-> generate_pdf_report()
```

4.2 Data fetch - fetch_company()

- Uses yfinance.Ticker: .info, financials, balance_sheet, cashflow, history, recommendations

- Retries with backoff (YFINANCE_FETCH_RETRIES) for rate limits
- Returns CompanyData dataclass

4.3 Valuation - three methods

Simon FCFE DCF (run_dcf): discount projected FCFE at cost of equity Ke; Gordon terminal value; mid-year convention; growth capped at 12%. Skipped for banks/insurance.

Comps (run_comps): sector peers from SECTOR_PEERS; median multiples (P/E, EV/EBITDA, P/S); implied price sanity 0.25x-3.0x current.

Analyst: Yahoo targetMeanPrice / recommendations.

Blend (build_valuation_summary): weights DCF 28% / Comps 44% / Analyst 28% on active methods; outlier filter; audit if upside > 30%; target cap 40%.

Rating: BUY/OVERWEIGHT, HOLD, SELL/UNDERWEIGHT from upside bands.

4.4 Top 20 batch - run_batch()

- build_summary_ticker_list: rank MEGA_CAP_UNIVERSE; force SPCX (SUMMARY_INCLUDE_TICKERS)
- build_pdf_ticker_list: top 20 PDFs + EXTRA_PDF_TICKERS (TSLA, BX) = 21 reports
- --refresh-stale: skip if period unchanged and last run < 7 days (.data_state.json)
- --force: regenerate all tickers
- Summary sorted by market_cap -> Top20_Summary.pdf + .csv

4.5 Key constants

Constant	Value	Meaning
TOP_US_SUMMARY_COUNT	20	Summary table rows
EXTRA_PDF_TICKERS	TSLA, BX	Extra PDFs
SUMMARY_INCLUDE_TICKERS	SPCX	Always in Top 20
STALE_MAX_AGE_DAYS	7	Stale refresh threshold
VALUATION_AUDIT_THRESHOLD_PCT	30	Audit trigger upside %
TARGET_SANITY_CAP_PCT	40	Max target vs current

4.6 CLI reference

```
python equity_research.py --batch-top20 --refresh-stale
python equity_research.py --batch-top20 --force
python equity_research.py --batch-top20 --weekly
python equity_research.py NVDA
```

5. dashboard_data.py

- fetch_macro_news(7): one high-impact headline per US Eastern day (past week, newest first)
- Impact score: macro keywords + premium sources + market-moving terms
- fetch_index_charts: ^GSPC, ^DJI, ^IXIC, ^RUT - 6M normalized sparklines
- Output: window.DASHBOARD_DATA in dashboard-data.js

6. sync_website.py

- Copy reports PDFs to public/reports and docs/reports; remove stale PDFs
- Top20_Summary.csv -> summary-data.js (window.SUMMARY_ROWS)
- build_dashboard_payload() -> dashboard-data.js
- Do not hand-edit JS data files; change Python and re-run sync

7. Frontend - public/index.html

- Static single-page site; no build step
- Loads summary-data.js + dashboard-data.js
- renderTable: sortable/searchable Top 20; renderReportButtons: PDF grid
- renderIndices + renderNews: market dashboard widgets
- Sections: #about #project #research #top20 #resume

8. GitHub Actions

File: .github/workflows/weekly-update.yml. Triggers: cron 0 14 * * 1 (Monday) and workflow_dispatch. Steps: checkout, pip install, batch + sync, git add reports/ public/ docs/, commit, push. Manual run supports Force full refresh.

9. Git workflow tips

- Commit source code (.py, index.html, workflows); let Actions generate PDFs and JS
- On JS merge conflicts: pick either side, then run sync_website.py
- After Actions pushes, Pull before local edits to avoid divergence

10. Common operations

Scenario	Steps
Change valuation	Edit equity_research.py -> force batch -> sync -> push
Add Top 20 ticker	SUMMARY_INCLUDE_TICKERS + MEGA_CAP_UNIVERSE
Change news logic	Edit dashboard_data.py -> sync_website.py
Change website UI	Edit public/index.html -> push
Update site now	Actions -> Run workflow

11. Dependencies

yfinance, pandas, numpy, reportlab, matplotlib, openpyxl - see requirements.txt

Regenerate this guide: python generate_project_guide_pdf.py